

marston_ward_assignment1

October 26, 2025

Part 1- Neural Network

```
[1]: import numpy as np
import os
import PIL
import PIL.Image
import tensorflow as tf
import tensorflow_datasets as tfds
import pathlib
import matplotlib.pyplot as plt

[2]: (X_train, y_train), (X_test, y_test) = tf.keras.datasets.fashion_mnist.
↳load_data()
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# plot 4 images as gray scale
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[y_train[i]])
plt.show()
```



```
[3]: # flatten 28*28 images to a 784 vector for each image
IMG_HEIGHT=28
IMG_WIDTH= 28
channels =1
```

```
[4]: # normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255

# One-hot encode the labels for F1Score compatibility
y_train = tf.keras.utils.to_categorical(y_train, num_classes=10)
y_test = tf.keras.utils.to_categorical(y_test, num_classes=10)
```

```
[5]: #Create the model:
# Input layer with shape=(IMG_HEIGHT, IMG_WIDTH) as the first layer
# Flatten() layer without the deprecated input_shape parameter
loss_fn = tf.keras.losses.CategoricalCrossentropy(from_logits=False)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(IMG_HEIGHT,
    ↪ IMG_WIDTH)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')])
model.compile(optimizer='adam', loss=loss_fn, metrics=['accuracy'])
```

```
[6]: # Train the model here
hist = model.fit(X_train, y_train, validation_split=0.2, epochs=20,
    ↪ batch_size=128)
```

```
Epoch 1/20
375/375          1s 969us/step -
accuracy: 0.7338 - loss: 0.7890 - val_accuracy: 0.8363 - val_loss: 0.4775
Epoch 2/20
375/375          1s 969us/step -
accuracy: 0.7338 - loss: 0.7890 - val_accuracy: 0.8363 - val_loss: 0.4775
Epoch 2/20
375/375          0s 917us/step -
accuracy: 0.8493 - loss: 0.4305 - val_accuracy: 0.8558 - val_loss: 0.4078
Epoch 3/20
375/375          0s 917us/step -
accuracy: 0.8493 - loss: 0.4305 - val_accuracy: 0.8558 - val_loss: 0.4078
Epoch 3/20
375/375          0s 815us/step -
accuracy: 0.8674 - loss: 0.3712 - val_accuracy: 0.8652 - val_loss: 0.3844
Epoch 4/20
375/375          0s 815us/step -
accuracy: 0.8674 - loss: 0.3712 - val_accuracy: 0.8652 - val_loss: 0.3844
Epoch 4/20
375/375          0s 816us/step -
accuracy: 0.8766 - loss: 0.3508 - val_accuracy: 0.8736 - val_loss: 0.3607
Epoch 5/20
375/375          0s 816us/step -
accuracy: 0.8766 - loss: 0.3508 - val_accuracy: 0.8736 - val_loss: 0.3607
Epoch 5/20
375/375          0s 807us/step -
accuracy: 0.8800 - loss: 0.3325 - val_accuracy: 0.8766 - val_loss: 0.3490
Epoch 6/20
375/375          0s 807us/step -
accuracy: 0.8800 - loss: 0.3325 - val_accuracy: 0.8766 - val_loss: 0.3490
Epoch 6/20
375/375          0s 815us/step -
```

accuracy: 0.8874 - loss: 0.3143 - val_accuracy: 0.8783 - val_loss: 0.3471
 Epoch 7/20
 375/375 0s 815us/step -
 accuracy: 0.8874 - loss: 0.3143 - val_accuracy: 0.8783 - val_loss: 0.3471
 Epoch 7/20
 375/375 0s 840us/step -
 accuracy: 0.8890 - loss: 0.3008 - val_accuracy: 0.8779 - val_loss: 0.3470
 Epoch 8/20
 375/375 0s 840us/step -
 accuracy: 0.8890 - loss: 0.3008 - val_accuracy: 0.8779 - val_loss: 0.3470
 Epoch 8/20
 375/375 0s 818us/step -
 accuracy: 0.8952 - loss: 0.2864 - val_accuracy: 0.8749 - val_loss: 0.3463
 Epoch 9/20
 375/375 0s 818us/step -
 accuracy: 0.8952 - loss: 0.2864 - val_accuracy: 0.8749 - val_loss: 0.3463
 Epoch 9/20
 375/375 0s 821us/step -
 accuracy: 0.8969 - loss: 0.2781 - val_accuracy: 0.8814 - val_loss: 0.3323
 Epoch 10/20
 375/375 0s 821us/step -
 accuracy: 0.8969 - loss: 0.2781 - val_accuracy: 0.8814 - val_loss: 0.3323
 Epoch 10/20
 375/375 0s 883us/step -
 accuracy: 0.9009 - loss: 0.2704 - val_accuracy: 0.8856 - val_loss: 0.3303
 Epoch 11/20
 375/375 0s 883us/step -
 accuracy: 0.9009 - loss: 0.2704 - val_accuracy: 0.8856 - val_loss: 0.3303
 Epoch 11/20
 375/375 0s 853us/step -
 accuracy: 0.9036 - loss: 0.2611 - val_accuracy: 0.8813 - val_loss: 0.3298
 Epoch 12/20
 375/375 0s 853us/step -
 accuracy: 0.9036 - loss: 0.2611 - val_accuracy: 0.8813 - val_loss: 0.3298
 Epoch 12/20
 375/375 0s 884us/step -
 accuracy: 0.9081 - loss: 0.2504 - val_accuracy: 0.8850 - val_loss: 0.3240
 Epoch 13/20
 375/375 0s 884us/step -
 accuracy: 0.9081 - loss: 0.2504 - val_accuracy: 0.8850 - val_loss: 0.3240
 Epoch 13/20
 375/375 0s 891us/step -
 accuracy: 0.9122 - loss: 0.2432 - val_accuracy: 0.8870 - val_loss: 0.3160
 Epoch 14/20
 375/375 0s 891us/step -
 accuracy: 0.9122 - loss: 0.2432 - val_accuracy: 0.8870 - val_loss: 0.3160
 Epoch 14/20
 375/375 0s 877us/step -

```

accuracy: 0.9142 - loss: 0.2362 - val_accuracy: 0.8860 - val_loss: 0.3235
Epoch 15/20
375/375          0s 877us/step -
accuracy: 0.9142 - loss: 0.2362 - val_accuracy: 0.8860 - val_loss: 0.3235
Epoch 15/20
375/375          0s 872us/step -
accuracy: 0.9144 - loss: 0.2284 - val_accuracy: 0.8872 - val_loss: 0.3135
Epoch 16/20
375/375          0s 872us/step -
accuracy: 0.9144 - loss: 0.2284 - val_accuracy: 0.8872 - val_loss: 0.3135
Epoch 16/20
375/375          0s 854us/step -
accuracy: 0.9207 - loss: 0.2213 - val_accuracy: 0.8865 - val_loss: 0.3208
Epoch 17/20
375/375          0s 854us/step -
accuracy: 0.9207 - loss: 0.2213 - val_accuracy: 0.8865 - val_loss: 0.3208
Epoch 17/20
375/375          0s 870us/step -
accuracy: 0.9171 - loss: 0.2175 - val_accuracy: 0.8901 - val_loss: 0.3222
Epoch 18/20
375/375          0s 870us/step -
accuracy: 0.9171 - loss: 0.2175 - val_accuracy: 0.8901 - val_loss: 0.3222
Epoch 18/20
375/375          0s 820us/step -
accuracy: 0.9229 - loss: 0.2104 - val_accuracy: 0.8942 - val_loss: 0.3067
Epoch 19/20
375/375          0s 820us/step -
accuracy: 0.9229 - loss: 0.2104 - val_accuracy: 0.8942 - val_loss: 0.3067
Epoch 19/20
375/375          0s 835us/step -
accuracy: 0.9250 - loss: 0.2060 - val_accuracy: 0.8878 - val_loss: 0.3226
Epoch 20/20
375/375          0s 835us/step -
accuracy: 0.9250 - loss: 0.2060 - val_accuracy: 0.8878 - val_loss: 0.3226
Epoch 20/20
375/375          0s 816us/step -
accuracy: 0.9282 - loss: 0.1968 - val_accuracy: 0.8924 - val_loss: 0.3079
375/375          0s 816us/step -
accuracy: 0.9282 - loss: 0.1968 - val_accuracy: 0.8924 - val_loss: 0.3079

```

```
[7]: hist.history
```

```

[7]: {'accuracy': [0.8032291531562805,
 0.8522291779518127,
 0.8676666617393494,
 0.8764374852180481,
 0.8811041712760925,

```

```
0.8880208134651184,  
0.8914791941642761,  
0.895145833492279,  
0.8969791531562805,  
0.9018124938011169,  
0.9057083129882812,  
0.906541645526886,  
0.9101250171661377,  
0.9131458401679993,  
0.9155625104904175,  
0.9179375171661377,  
0.9183124899864197,  
0.92208331823349,  
0.9233124852180481,  
0.9257916808128357],  
'loss': [0.5764191746711731,  
0.4198676645755768,  
0.3752998411655426,  
0.34693974256515503,  
0.32807815074920654,  
0.3108524978160858,  
0.298229455947876,  
0.2863965928554535,  
0.27952155470848083,  
0.2680234909057617,  
0.2567472457885742,  
0.2529747188091278,  
0.24737851321697235,  
0.23791639506816864,  
0.23008206486701965,  
0.22289492189884186,  
0.21808762848377228,  
0.21206487715244293,  
0.2075471580028534,  
0.20239581167697906],  
'val_accuracy': [0.8363333344459534,  
0.8558333516120911,  
0.8652499914169312,  
0.8735833168029785,  
0.8765833377838135,  
0.878333330154419,  
0.877916693687439,  
0.874916672706604,  
0.8814166784286499,  
0.8855833411216736,  
0.8813333511352539,  
0.8849999904632568,
```

```

0.8870000243186951,
0.8859999775886536,
0.8871666789054871,
0.8865000009536743,
0.8900833129882812,
0.8942499756813049,
0.8878333568572998,
0.8924166560173035],
'val_loss': [0.4775412976741791,
0.40780705213546753,
0.38437771797180176,
0.36065882444381714,
0.3489731550216675,
0.3470900058746338,
0.3469929099082947,
0.3462677299976349,
0.3322739899158478,
0.33028554916381836,
0.3298058807849884,
0.3239690661430359,
0.31597691774368286,
0.3234546184539795,
0.3134985864162445,
0.32080569863319397,
0.322223222255707,
0.30667927861213684,
0.32255589962005615,
0.30791398882865906]]}

```

```

[8]: plt.figure(figsize=(12, 5))

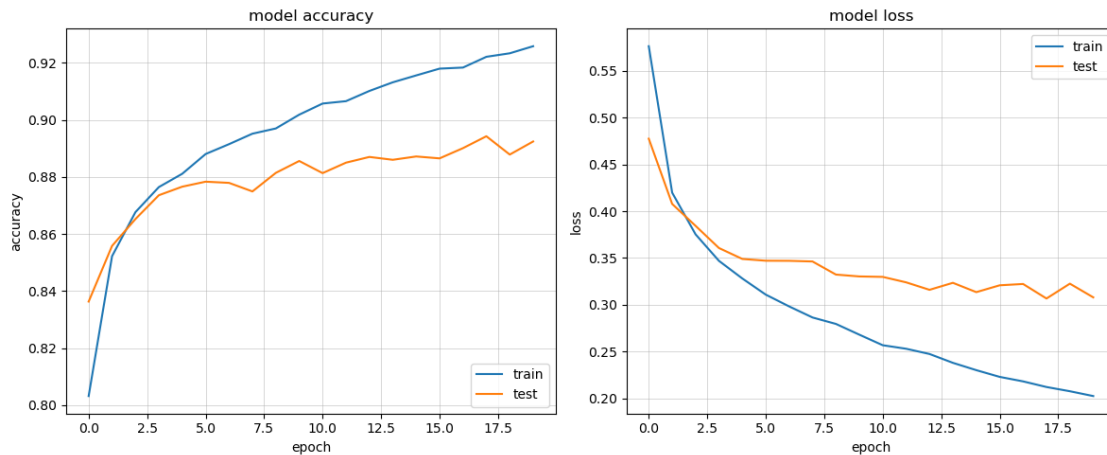
plt.subplot(1,2,1)
plt.plot(hist.history['accuracy'])
plt.plot(hist.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.grid(linewidth=0.5, alpha=0.7)
plt.legend(['train', 'test'], loc='lower right')

plt.subplot(1,2,2)
plt.plot(hist.history['loss'])
plt.plot(hist.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.grid(linewidth=0.5, alpha=0.7)

```

```
plt.legend(['train', 'test'], loc='upper right')

plt.tight_layout()
plt.show()
```



1 Discussion

The model is a simple feedforward neural network with one hidden layer. It uses the ReLU activation function in the hidden layer and softmax activation in the output layer for multi-class classification, in this case, 10. The model is trained using the Adam optimizer and sparse categorical cross-entropy loss function, which is suitable for integer-labeled multi-class classification tasks. The only metric generated was accuracy. The training process includes a validation split to monitor the model's performance on unseen data during training. The accuracy and loss plots indicate how well the model learns over epochs, showing improvements in both metrics.

Part 2- Image Processing

Load the Flower photo dataset from tensorflow repository

```
[9]: dataset_url = "https://storage.googleapis.com/download.tensorflow.org/
    ↪example_images/flower_photos.tgz"
    data_dir = tf.keras.utils.get_file(origin=dataset_url,
                                       fname='flower_photos',
                                       untar=True)
    data_dir = pathlib.Path(data_dir)
```

a) How many images we can find in this dataset?

```
[11]: data_dir_sub = (data_dir / 'flower_photos').resolve()
    images = list(data_dir_sub.rglob('**/*.jpg')) #This will count all the file_
    ↪with extension of jpg- You have to modify this part
    print(len(images))
```

3670

b) The list of subfolders are: * daisy * dandelion * roses * sunflowers * tulips

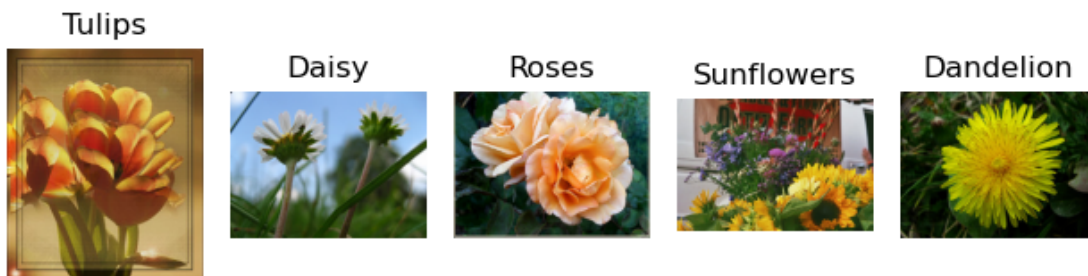
You can look into any of the subfolders to see images stored over there. You can look into the folder using: `data_dir.glob('tulips/*')` For this part use Pillow (PIL) to show at least one flower from each subfolder

```
[12]: # There are 5 subfolders in the dataset
select_images = {'tulips': None, 'daisy': None, 'roses': None, 'sunflowers': None,
                 'dandelion': None}
for flower in select_images.keys():
    print(f"{flower} images")
    ddir = (data_dir_sub / flower).resolve()
    imgs = list(ddir.rglob('*.jpg'))
    if imgs:
        img_path = list(imgs)[2]
        select_images[flower] = PIL.Image.open(img_path)
    else:
        print(f"No images found for {flower}")

for i, (flower_name, img) in enumerate(select_images.items()):
    if img:
        # Create a subplot for each image (1 row, 5 columns, current index + 1)
        plt.subplot(1, 5, i + 1)
        plt.imshow(img)
        plt.title(flower_name.capitalize())
        plt.axis('off')

plt.tight_layout()
plt.show()
```

```
tulips images
daisy images
roses images
sunflowers images
dandelion images
```



c) Use Keras to resize all the images into same dimension 180x180

```
[ ]: batch_size = 32
img_height = 180
img_width = 180

train_ds = tf.keras.utils.image_dataset_from_directory(
    directory=data_dir_sub,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size
)
```

Found 3670 files belonging to 5 classes.

Using 734 files for training.

Using 734 files for training.

```
[14]: # Use the same strategy to create validation data, this time from validation
      ↪subset
val_ds = tf.keras.utils.image_dataset_from_directory(
    directory=data_dir_sub,
    validation_split=0.2,
    subset="validation",
    seed=123,
    # Resize
    image_size=(img_height, img_width),
    batch_size=batch_size
)
```

Found 3670 files belonging to 5 classes.

Using 734 files for validation.

Using 734 files for validation.

d) You can use `train_ds.class_names` command to get the list of labels. Write a code to randomly show 9 images from training data while printing their label on top of the image.

```
[15]: class_names = train_ds.class_names
plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
```

2025-10-26 17:19:51.234471: I tensorflow/core/framework/local_rendezvous.cc:407]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

roses



dandelion



tulips



sunflowers



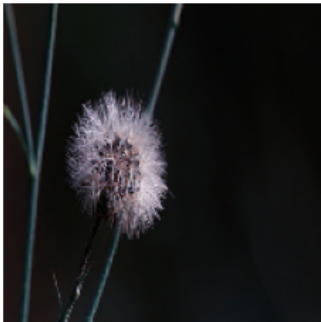
dandelion



roses



dandelion



roses



tulips



1.0.1 Difficulties in the ML properly indentifying these images

The images are take from different perspectives: some near (close up) and some far away. This will cause a problem because some strongly identifying features are enhanced in a close up but disappear when further away. In another image the sun is low in the sky putting a dandelion in the shadows but outlining the stem of the flower inside the sphere of seeds around it. IN some cases the butterfly hides parts of the flower and in another a flower is shown in different stages of development (which look nothing alike if you are not aware of the flowers life cycle). Moreover, some images have dominate features that is not a flower and might confuse the model. Finally, some images are sharpe and some are a bit blurry. All of these issues affect the data's health, which is not uniform, which will affect the computer vision's ability to mathematically identify flowers when predicting.

e - Now, we want to use preprocessing package in Keras to apply different filters to the image. Apply the following procedures to image data: - Rescale the image by dividing by 255 - Shear the image 20% - Zoom the image 20% - Horizontally flip the images

```
[23]: # Modern preprocessing pipeline (replacement for deprecated ImageDataGenerator)
# Using the modern preprocessing/augmentation pipeline we use for the flower_
↳dataset.
import keras
from keras import layers
AUTOTUNE = tf.data.AUTOTUNE # lets TensorFlow pick optimal parallelism/
↳prefetching.

# Data augmentation and rescaling
data_augmentation = keras.Sequential([
    layers.Rescaling(1.0/255), # scales from 0-255 to 0-1
    layers.RandomFlip("horizontal"),
    layers.RandomZoom(0.2), # zoom to about 20%
    layers.RandomRotation(0.05), # small rotation as a proxy for shear
])

# Apply to datasets
train_ds = train_ds.map(lambda x, y: (data_augmentation(x, training=True), y),
↳num_parallel_calls=AUTOTUNE)
val_ds = val_ds.map(lambda x, y: (layers.Rescaling(1./255)(x), y),
↳num_parallel_calls=AUTOTUNE)

# Performance: cache and prefetch
train_ds = train_ds.cache().prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

Part 3- OPENCV- Now use opencv for preprocessing. Show first 9 images in dataset using Opencv. Before showing each image, resize the images to 180x180.

```
[24]: # Repeat same preprocessing but this time using OpenCV. Show first 9 images in
↳dataset using OpenCV.
import cv2
from pathlib import Path

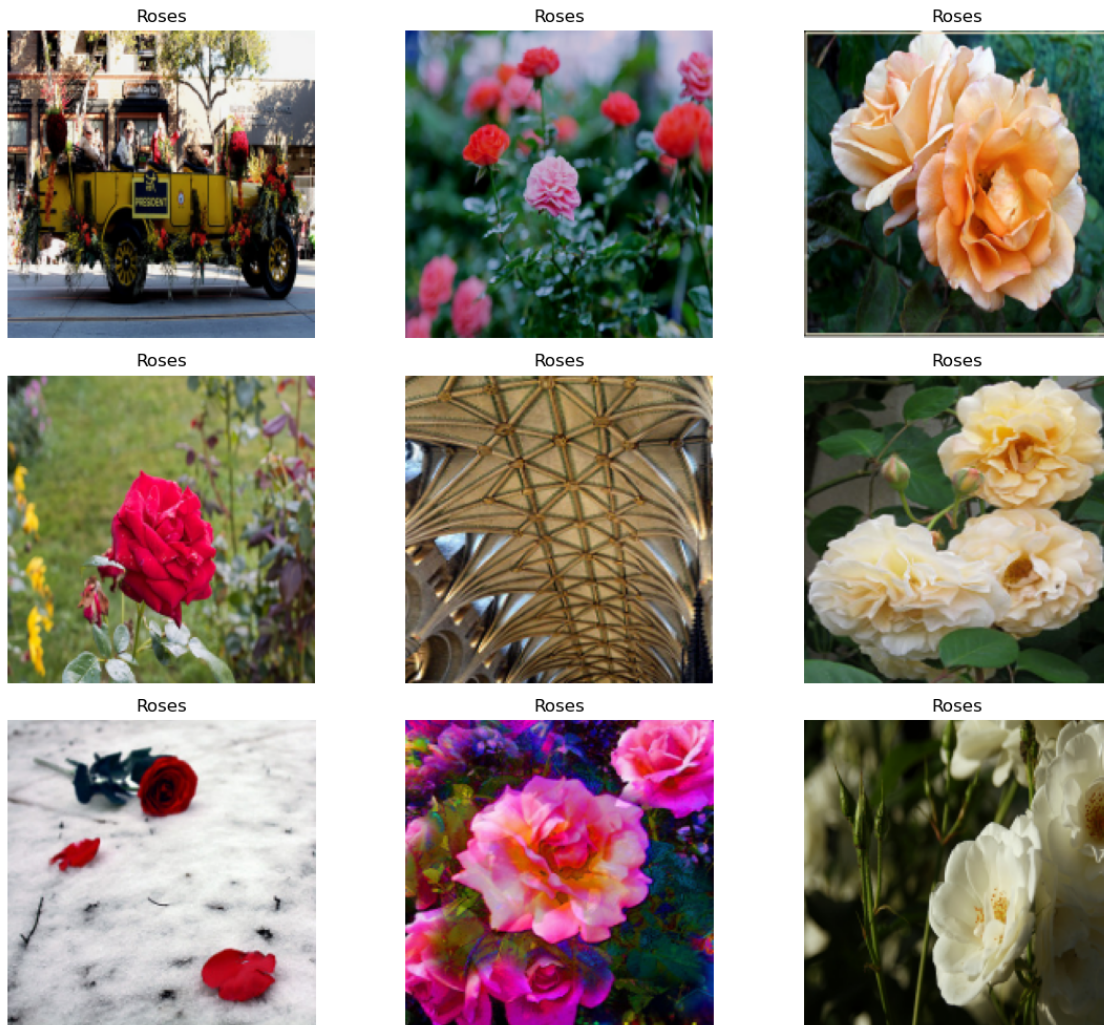
# Collect images from the flower_photos subdirectory
list_of_images = list((data_dir / 'flower_photos').glob('*/*.jpg'))

plt.figure(figsize=(12, 10))
for i in range(9):
    img_path = Path(list_of_images[i])
    img = cv2.imread(str(img_path)) # BGR format
    if img is None:
        continue
    # Resize to 180x180 and convert to RGB for matplotlib
```

```
img_resized = cv2.resize(img, (180, 180), interpolation=cv2.INTER_AREA)
img_rgb = cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB)
```

```
ax = plt.subplot(3, 3, i + 1)
plt.imshow(img_rgb)
plt.title(img_path.parent.name.capitalize())
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```



OpenCV uses BGR as its default colour order for images, matplotlib uses RGB. When you display an image loaded with OpenCv in matplotlib the channels will be back to front. The easiest way of fixing this is to use OpenCV to explicitly convert it back to RGB, much like you do when creating the greyscale image. `RGB_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

```
[25]: # Demonstrate BGR vs RGB display and the proper conversion
import cv2
from pathlib import Path

# Pick one sample image
candidates = list((data_dir / 'flower_photos').glob('*/*.jpg'))
if not candidates:
    raise RuntimeError("No images found under flower_photos. Run the download/
↳setup cells above.")

img_path = Path(candidates[0])
img_bgr = cv2.imread(str(img_path)) # OpenCV reads in BGR
if img_bgr is None:
    raise RuntimeError(f"Failed to read image: {img_path}")

# Wrong display (treating BGR as RGB) vs Correct display (after conversion)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img_bgr) # Incorrect colors because it's BGR interpreted as RGB
plt.title("BGR shown as RGB (wrong)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(img_rgb)
plt.title("Converted to RGB (correct)")
plt.axis("off")

plt.suptitle(f"Example: {img_path.parent.name}")
plt.tight_layout()
plt.show()
```

Example: roses

